Pilas

Esta es la primera estructura lógicas que veremos en el curso. Nos referimos a estructura lógica porque no vamos a emplear nuevas palabras reservadas ni alguna otra funcionalidad de C, sino que vamos a desarrollar una estructura nueva y la vamos a manipular a partir de principios lógicos en su definición.

Decimos que tenemos una pila cuando contamos con una estructura que almacena datos de tal manera que estos datos se apilan uno encima de otro y únicamente podemos acceder al último dato ingresado. A partir de ahí podemos ir sacando el dato a continuación así hasta llegar al último dato en la pila o sea el primero que se ingresó. Con esta lógica únicamente podemos acceder a los datos del fondo de la pila sacando los datos previamente ingresados.

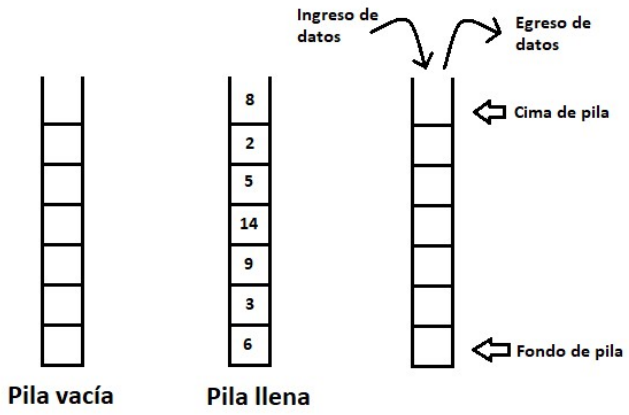
Pensemos en la pila como un contenedor vertical en cual los datos ingresan y egresan por el mismo lugar, esta lógica responde al principio LIFO (Last In, First Out), lo que significa que el último elemento en entrar es el primero en salir. En otras palabras, los elementos se agregan y se eliminan de la pila por el mismo extremo, conocido como el tope o cima de la pila.

Utilidades de una pila

- Evaluación de expresiones matemáticas y conversión entre notaciones (postfija, infija, prefija).
- Implementación de funcionalidades de deshacer y rehacer (undo/redo) en editores de texto.
- Control de recursividad en llamadas a funciones.
- Gestión de la memoria en sistemas operativos.

A fines prácticos en este curso veremos el desarrollo de una pila de enteros, pero podemos pensar en pilas de cualquier tipo de datos o datos más complejos tales como ubicaciones de posiciones en un disco rígido o direcciones de memoria.

Pensemos en la pila de enteros de la siguiente manera:



La pila la podemos pensar como un contenedor vertical, los datos ingresan por la cima del contenedor y van al fondo a ocupar el último lugar disponible. Sobre la izquierda vemos el contenedor vacío haciendo referencia a una pila vacía, a medida que ingresamos los datos, estos van ocupando un lugar hasta que la pila se llena. Sobre la derecha vemos el ingreso y egreso de datos. Pero volvamos al dibujo del medio, si quisiéramos acceder al número 3 previamente debemos quitar el 8, 2, 5, 14 y 9 en ese orden.

Operaciones básicas de una pila

- Push/Agregar: Inserta un elemento en la cima de la pila.
- Pop/Quitar: Remueve y devuelve el elemento que está en la cima.
- IsTop/Pila Llena: Verifica si pila alcanzó el tope de capacidad.
- IsEmpty/Pila Vacía: Verifica si la pila no tiene elementos.
- Inicializar pila: le asigna valores iniciales para manipular la pila

Codificando una pila

```
#define stackSize 5

typedef struct pila
{
    int elementos[stackSize];
    int tope;
}tpila;
```

En primer lugar, creamos la estructura. Los atributos internos son un vector que va a contener los datos de la pila, el tamaño del vector lo definimos con la constante ***stackSize***, y una variable entera tope que nos va a controlar las posiciones del vector. A tener en cuenta que durante la manipulación de una pila solamente podremos utilizar las funciones que creamos a continuación de operaciones básicas. Estas son las únicas que van a manipular el vector, la idea de esta estructura lógica es manipularla únicamente con estas funciones sin acceder al vector interno.

```
void incializarPila(tpila *p)
{
    p->tope=-1;
}
```

La función inicializar pila es necesaria para establecer el valor inicial de tope, el cual fijamos en -1. Este valor hace referencia a una pila vacía y a medida que vayamos agregando datos vamos modificando este valor que nos va a servir de índice del vector.

```
int pilaLlena(tpila *p)
{
    if(p->tope==stackSize-1)
        return 1;
    else
        return 0;
}
```

```
int pilaVacia(tpila *p)
{
    if(p->tope==-1)
        return 1;
    else
        return 0;
}
```

En este caso ambas funciones sirven de verificación del estado de la pila. Un índice con el valor igual al tamaño del vector indica una pila llena, y el mismo índice en -1 indica una pila vacía. Estas dos funciones sirven a modo de verificación booleana para las funciones que crearemos a continuación.

Agregando datos

Para agregar un dato a una pila es primordial verificar que exista lugar para el ingreso del mismo, para ello recurrimos a la función anterior. Una vez verificada la disponibilidad procedemos a modificar el índice y luego insertar el dato

```
void agregarPila(tpila *p,int dato)
{
    if(!pilaLlena(p))
    {
        p->tope++;
        p->elementos[p->tope]=dato;
    }
    else
    {
        printf("Pila llena\n");
    }
}
```

Quitando datos

El proceso de quita de datos es inverso y similar al de agregar, pero con la diferencia que para poder quitar un dato previamente la pila no debe estar vacía. Luego de eso podemos quitar el dato y modificar el índice.

```
int quitarPila(tpila *p)
{
    if(!pilaVacia(p))
    {
        int dato;
        dato=p->elementos[p->tope];
        p->tope--;
        return dato;
    }
    else
    {
        printf("Pila sin datos\n");
    }
}
```

A partir de este punto con estas funciones básicas podemos empezar a trabajar con la pila sin acceder a los elementos internos de la misma. Veamos un caso en donde queremos mostrar todos los elementos que contiene una pila. Si partimos del principio lógico que no podemos acceder al vector, debemos pensar en utilizar las funciones creadas para manipular la pila. Entonces deberíamos ir quitando los datos mientras la pila no esté vacía, mostrar los datos, guardar estos datos para no perderlos y luego devolverlos a la pila original para que no quede alterada.

Mostrando los datos de una pila

```
void recorrePila(tpila *p)
{
    tpila auxpila;
    inicializarPila(&auxpila);
    while(!pilaVacia(p))
    {
        int dato=quitarPila(p);
        printf("%d\n",dato);
        agregarPila(&auxpila,dato);
    }
    while(!pilaVacia(&auxpila))
    {
        agregarPila(p,quitarPila(&auxpila));
    }
}
```

Ejemplo práctico de aplicación

```
int main()
{
    tpila nueva;
    inicializarPila(&nueva);
    agregarPila(&nueva,5);
    agregarPila(&nueva,6);
    agregarPila(&nueva,12);
    agregarPila(&nueva,3);
    agregarPila(&nueva,9);
    printf("Mostrando valores\n");
    recorrePila(&nueva);
    printf("Quito ultimo valor\n");
    quitarPila(&nueva);
    printf("Mostrando valores nuevamente\n");
    recorrePila(&nueva);

    return 0;
}
```